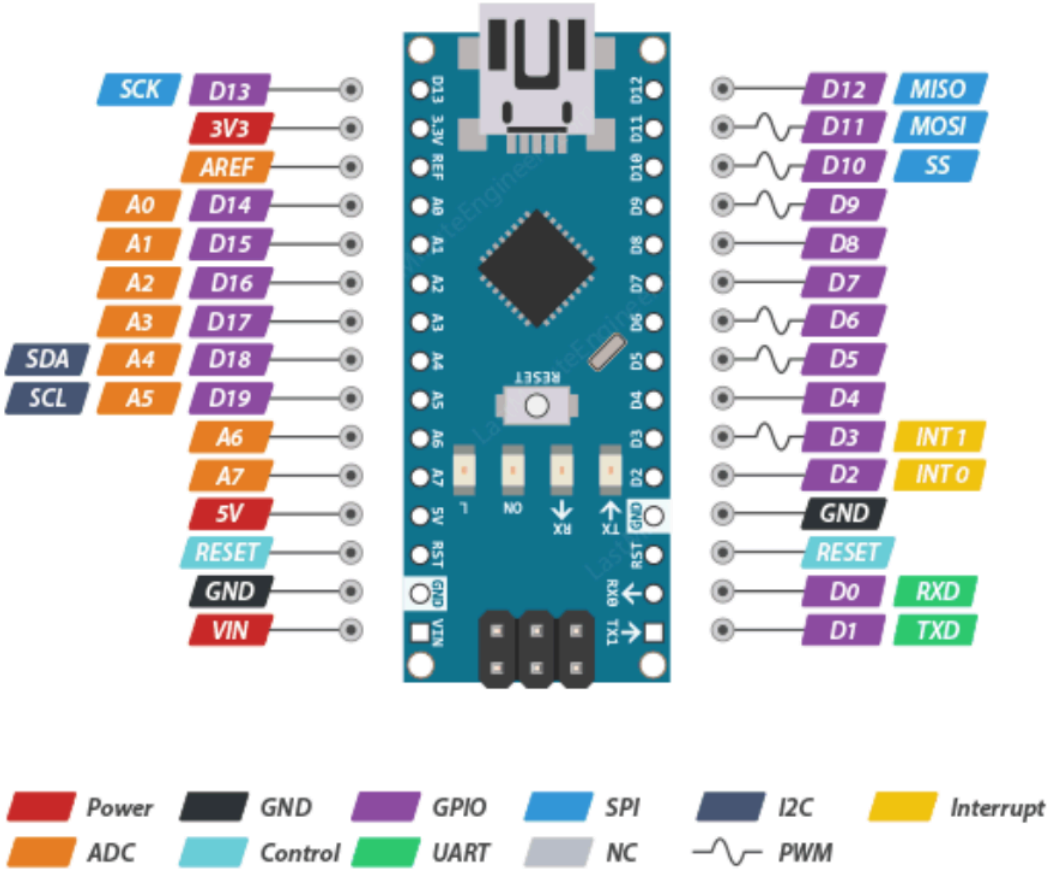
Arduino NANO

The Arduino Nano is a compact, breadboard-friendly microcontroller board based on the ATmega328P chip.

- **Core Logic:** It offers the exact same functionality as the Arduino Uno but in a much smaller form factor, making it ideal for space-constrained projects like robotics or custom PCB designs.
- **Operating Voltage:** 5V
- **Clock Speed:** 16 MHz
- **Interface:** Uses a Mini-B USB cable for programming and serial monitoring instead of the standard Type-B.



ARDUINO NANO

The Nano features 30 pins in total, designed to plug directly into a breadboard. The pins are logically grouped into **power, analog,** and **digital functions**, with many pins serving multiple specialized roles depending on how you configure them in your code.

Digital I/O Pins (D0 - D13)

1. Overview & Functionality

What they are: These 14 pins (labeled **D0 to D13**) are the primary way your Arduino interacts with other hardware using binary logic. They deal exclusively in two states: **HIGH** (ON) or **LOW** (OFF).
Two-Way Street: They can be configured in your code to act as either:

- **Inputs:** To read incoming digital signals (e.g., checking if a push-button is pressed or a magnetic switch is triggered).
- **Outputs:** To send digital signals out (e.g., turning on LEDs, driving motors, or activating relays).

2. Electrical Specifications

Logic Level: These pins operate strictly at a **5V logic level**.

Current Limits (Crucial): Each pin can safely source (supply) or sink (absorb) a maximum of **20 mA** of current. Exceeding this limit can permanently damage the microcontroller pin.

3. Coding Syntax & Application

To use a digital pin, you must first configure its behavior in your `setup()` function, and then command or read it in your `loop()`.

Configuring as an INPUT:

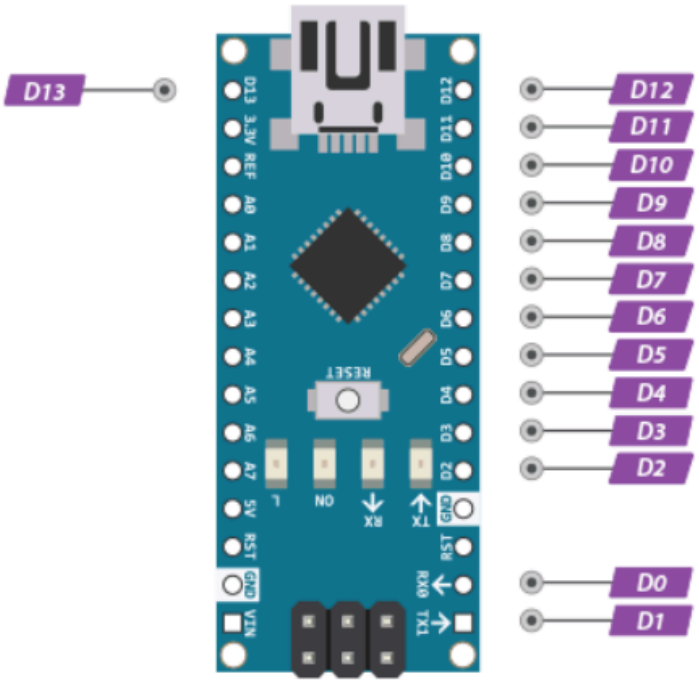
Setup: Use `pinMode(pin, INPUT)` to prepare the pin to read.

Read: Use `digitalRead(pin)` to check the pin's state. It will return either `HIGH` or `LOW`.

Configuring as an OUTPUT:

Setup: Use `pinMode(pin, OUTPUT)` to prepare the pin to send voltage.

Command: Use `digitalWrite(pin, HIGH)` to output 5V, or `digitalWrite(pin, LOW)` to drop the output to 0V.

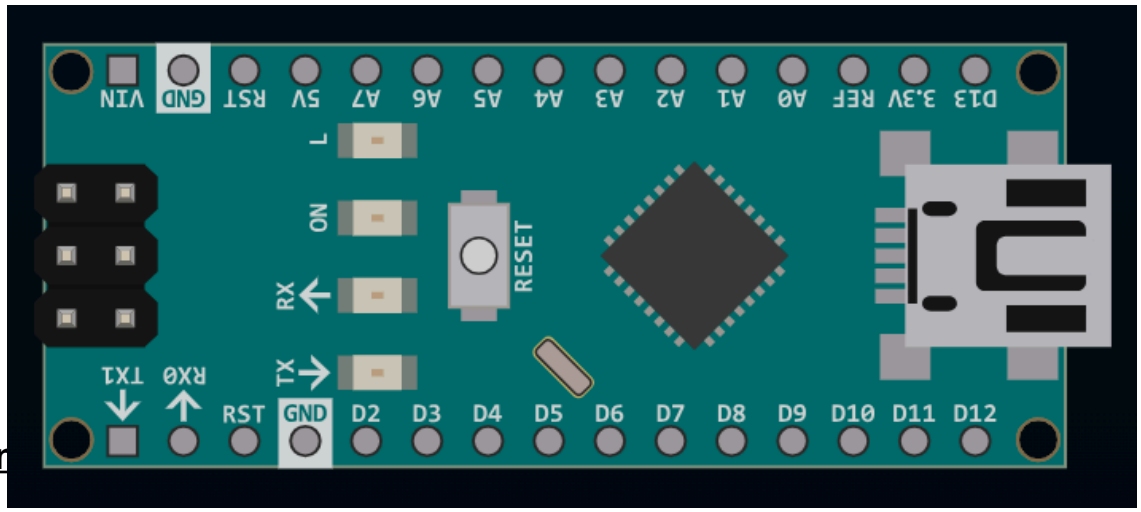


Onboard LED

The Arduino Nano features four tiny Surface Mount Device (SMD) LEDs right on the board. They act as immediate hardware diagnostics to let you know what the board is doing without needing a serial monitor.

The Four LEDs Explained

- **PWR (Power):** * **Function:** Indicates that the board is receiving power.
  - **Logic:** It is hardwired to the 5V line. If this LED is off while the board is plugged in, you either have a dead board, a bad USB cable, or a short circuit on your breadboard pulling the voltage down.
- **L (Built-in LED):**
  - **Function:** The only LED you can control directly via code.
  - **Logic:** It is internally connected to Digital Pin 13 through a small current-limiting resistor.
  - **Code:** Turning digitalWrite(13, HIGH) turns the LED on. It is perfect for quickly testing if your code is uploading and running properly (like the classic "Blink" sketch).
- **TX (Transmit):**
  - **Function:** Flashes when the microcontroller is sending data to your PC.
  - **Logic:** Controlled by the USB-to-Serial chip. You will see this blink furiously when you upload code or use Serial.println().
- **RX (Receive):**
  - **Function:** Flashes when the microcontroller is receiving data from your PC.
  - **Logic:** Also controlled by the USB-to-Serial chip. Blinks during code uploads or when you type commands into the Serial Monitor.

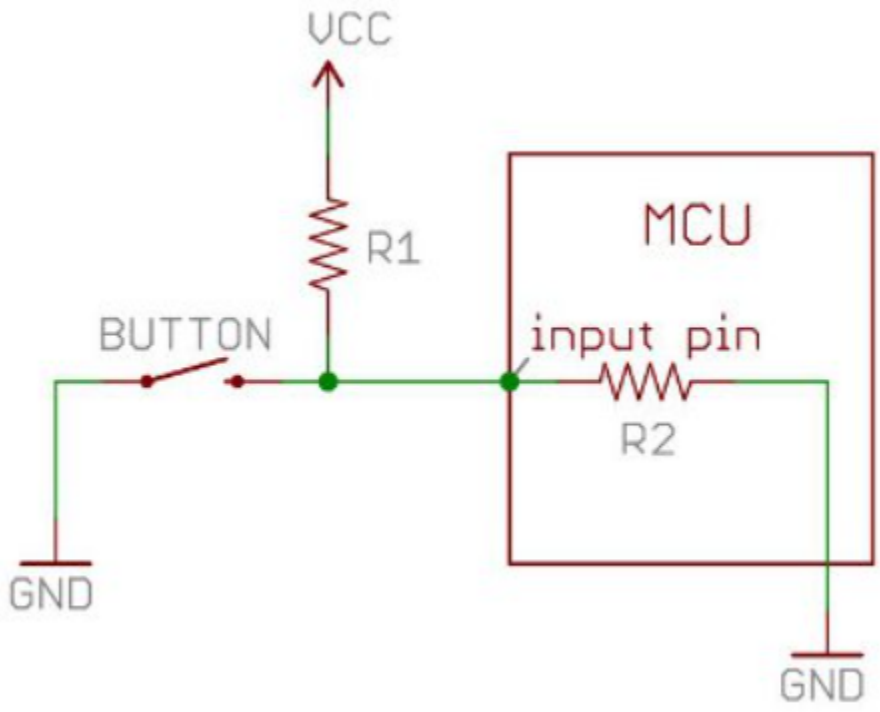


Intern

Every digital pin is equipped with an internal pull-up resistor that can be activated within INPUT mode by calling pinMode(pin, INPUT_PULLUP). This internal component weakly pulls the pin to

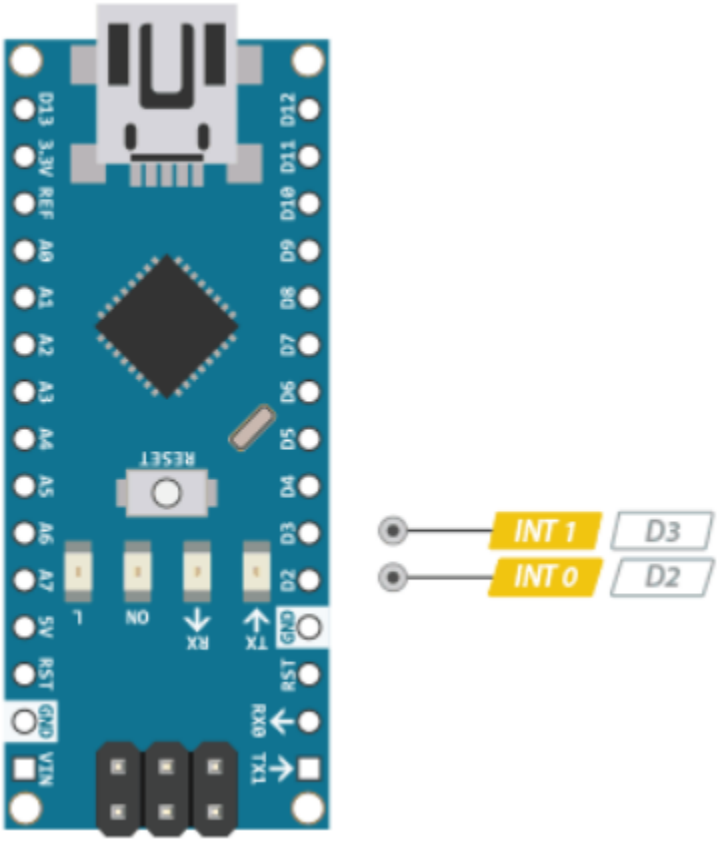
a HIGH logic state when no external connection is present, thereby preventing the pin from "floating" between states due to ambient electrical noise.

The following schematic illustrates the **internal pull-up resistor logic**, detailing how the integrated 20k-50k ohm component bridges the digital pin to the 5V rail within the microcontroller. This configuration ensures the pin remains in a **HIGH state** by default, until an external switch is triggered to pull the signal to ground.



Interrupt Pins

- **Pins:** D2 (External Interrupt 0) and D3 (External Interrupt 1).
- **Function:** These pins are essential for executing time-critical hardware tasks without the need to halt the main execution loop. Rather than utilizing polling to monitor a sensor, you can implement attachInterrupt() to trigger a specific block of code the moment a state change is detected—ideal for capturing high-speed pulses from components like vibration sensors.



PWM

- **Definition:** Pulse Width Modulation (PWM) enables digital pins to simulate analog output by rapidly oscillating between HIGH and LOW states, effectively generating a square wave signal.
- **Duty Cycle:** This represents the ratio of "ON" time relative to the total period of the signal. By modulating the duty cycle, you can adjust the average voltage delivered to an external component.
- **Applications:** This technique is ideally suited for dimming LEDs, regulating motor speeds, or generating basic audio frequencies.

Hardware Pins & Identification

- **Available Pins:** The Nano provides 6 PWM-capable digital pins: **D3, D5, D6, D9, D10, and D11**.
- **Visual Marker:** On the physical board, these pins are distinguished by a tilde (**~**) symbol located adjacent to their respective pin numbers.

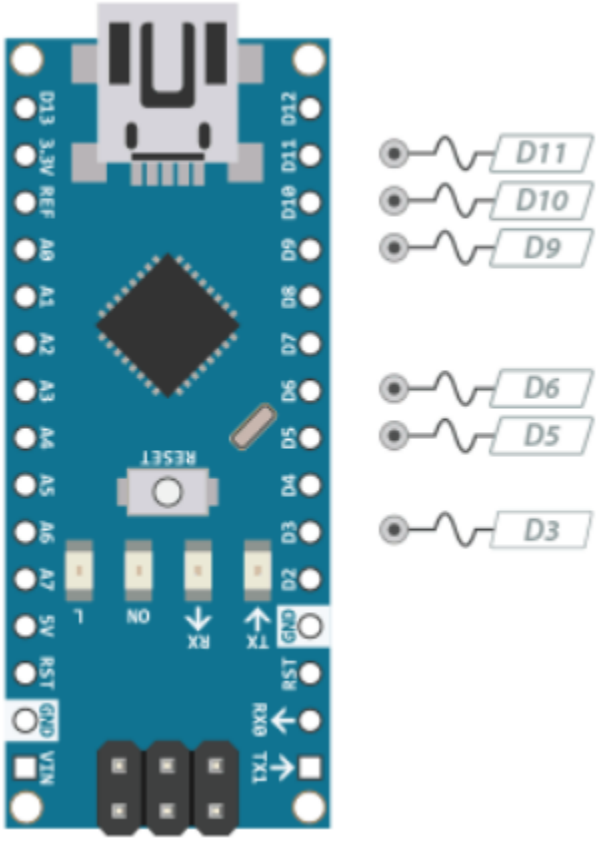
Signal Resolution & Coding Syntax

- **8-Bit Resolution:** The PWM signal is controlled by an 8-bit scale, facilitating 256 discrete intensity levels ranging from **0 to 255**.
- **Initialization:** To prepare a pin, define it as an output within the setup block using pinMode(pin, OUTPUT).
- **Command:** Execute the analogWrite(pin, value) function to drive the PWM signal to the pin.
    - analogWrite(pin, 0): Results in a **0%** duty cycle (Constant LOW / 0V).
    - analogWrite(pin, 127): Results in a **~50%** duty cycle (Approximate half voltage).
    - analogWrite(pin, 255): Results in a **100%** duty cycle (Constant HIGH / 5V).

Frequencies & Hardware Timers

- **Default Frequencies:**
    - Standard Pins (D3, D9, D10, D11): Operate at **~490 Hz**
    - Fast Pins (D5, D6): Operate at **~980 Hz**
- **Timer Associations:** Each PWM pin is mapped to a specific hardware timer on the ATmega328P:
    - **Timer 0:** Assigned to D5 and D6
    - **Timer 1:** Assigned to D9 and D10
    - **Timer 2:** Assigned to D3 and D11

- **Critical Warning:** Manipulating internal registers for custom frequencies is possible; however, altering **Timer 0** will compromise fundamental Arduino timing functions like delay() and millis().

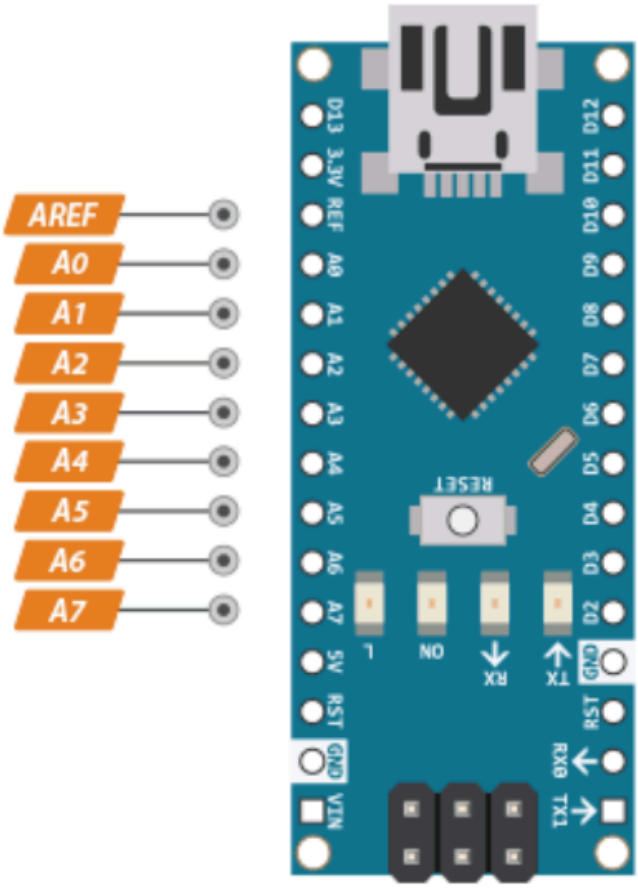


ADC Pins

Analog-to-Digital Converter (ADC) Capabilities

1. Resolution & Voltage Mapping

> **10-Bit ADC:** The Nano's internal Analog-to-Digital Converter operates at a 10-bit resolution, meaning it slices the incoming analog signal into **1024 discrete levels**



.

(The Arduino Nano has 8 analog input pins, labeled A0 through A7.)

Analog input pins on the Arduino Nano are designed to read continuous analog signals. These signals often come from sensors that measure physical quantities like temperature, light intensity, or pressure. The Arduino Nano converts these analog signals into digital values using an Analog-to-Digital Converter (ADC), allowing you to process and analyze the data within your Arduino code.

The Range: It translates real-world input voltages (from **0V to 5V**) into a digital integer range of **0 to 1023**.

Precision: This mapping results in a resolution step of roughly **4.9 mV** per unit (5V / 1024 steps).

Pro Tip: While 4.9 mV precision is plenty for standard DIY projects, you will need to wire up an external, higher-resolution ADC module (like a 12-bit or 16-bit ADS1115) if your project requires hyper-precise scientific measurements.

2. Coding Syntax & Application

- **The Command:** To capture this data in your code, use the analogRead(pin) function.
- **The Output:** The function returns an integer between 0 and 1023.
- **Example:** If your sensor outputs exactly **2.5V** (exactly half of the 5V maximum), analogRead() will return a value of roughly **512**.

3. Pin Versatility & Hardware Limits

- **Dual-Purpose Pins (A0 to A5):** If your project runs out of standard digital pins, A0 through A5 can act as digital I/O. When using them this way in your code, they are officially referenced as digital **pins 14 through 19**.
- **Strictly Analog Pins (A6 and A7):** These two pins are unique. They are wired *exclusively* to the multiplexer of the ADC and completely lack the internal circuitry required to act as digital pins. They are read-only analog pins.

Changing Reference Voltage

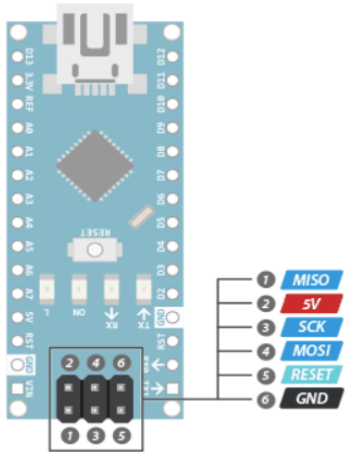
While the ADC normally maps signals within a 0V to 5V range, this window can be customized via the **AREF pin** and the analogReference(type) function. The available type configurations include:

- **DEFAULT:** Configures the system to use the standard 5V analog reference.
- **INTERNAL:** Engages an onboard reference voltage of 1.1V for higher sensitivity at lower voltage ranges.
- **EXTERNAL:** Enables the ADC to utilize a custom reference voltage provided through the AREF pin.

ICSP Header Pins

1. Overview & Physical Layout

- **What it is:** ICSP stands for **In-Circuit Serial Programming**.
- **Layout:** It is a dedicated block of 6 pins arranged in a strict **2x3 configuration** located on the top of the Nano board.
- **The Pins Included:** It houses VCC, GND, RESET, and the three core SPI data lines (MISO, MOSI, and SCK).



2. Primary Use Case: Direct Programming

- **The Function:** It allows you to push code directly into the ATmega328P's memory using an external hardware programmer (like a dedicated AVR ISP device, or simply a second Arduino board configured as an "Arduino as ISP").
- **Why use it? * Recovery:** If the board's bootloader becomes corrupted and standard USB uploads fail, you use these pins to "burn" a fresh bootloader onto the chip.
    - **Memory Optimization:** By bypassing the bootloader completely and uploading via ICSP, you can reclaim the ~0.5 KB of flash memory normally reserved for the bootloader for your own massive sketches.

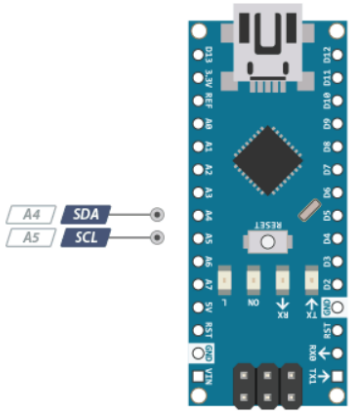
3. Secondary Use Case: SPI Communication

- **The Function:** Because the ICSP pins map directly to the microcontroller's hardware SPI bus, this header provides a convenient connection point for SPI-based peripheral devices.
- **The Data Lines:** It gives you direct access to **MISO** (Master In Slave Out), **MOSI** (Master Out Slave In), and **SCK** (Serial Clock).
- **Hardware Caveat (Crucial):** While it has the data and clock lines, the **SS (Slave Select)** pin is *not* included in this 2x3 block. If you wire an SD card module or sensor to this header, you must still run a separate jumper wire to a standard digital pin (typically D10) to manually control the Slave Select logic.

I2C (Inter-Integrated Circuit) Pins

- **Pins:** A4 (SDA - Serial Data) and A5 (SCL - Serial Clock).

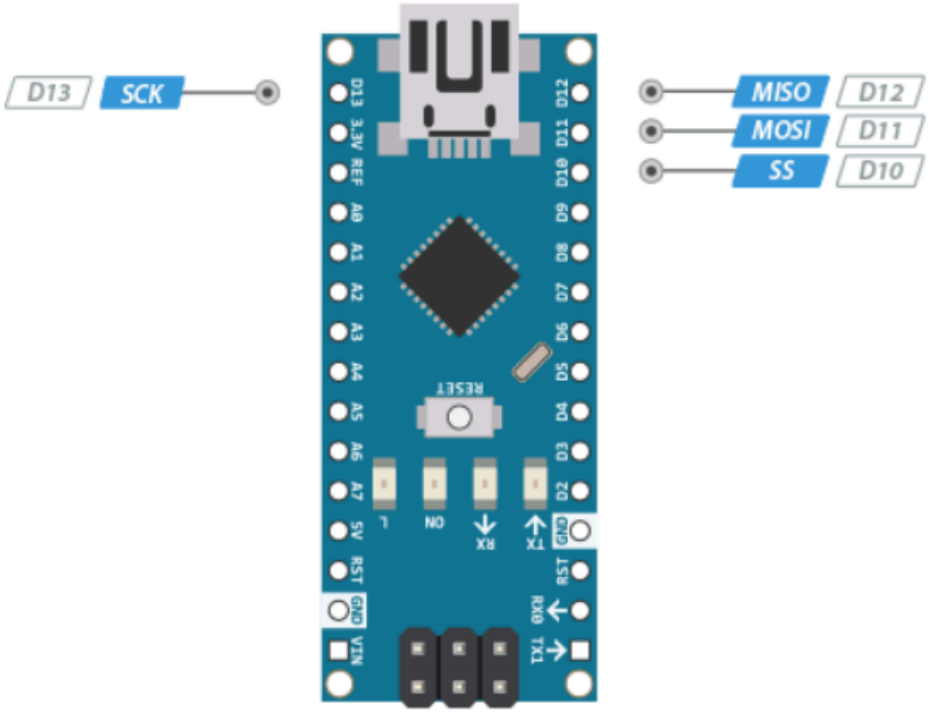
- **Function:** A synchronous, multi-master, multi-slave communication protocol.
- **Application:** Highly efficient for connecting multiple peripheral devices (like LCDs, OLED screens, or advanced sensor modules) to the microcontroller using only two wires.



SPI (Serial Peripheral Interface) Pins

1. Overview & Working Principle

- **What it is:** SPI is a high-speed, synchronous communication protocol primarily utilized for rapid, short-distance data exchange between the microcontroller (acting as the "Master") and various peripheral components (the "Slaves").
- **Applications:** This protocol is the standard choice for data-intensive or speed-critical hardware like SD card modules, TFT display screens, and RFID readers.



2. Hardware I

to facilitate this

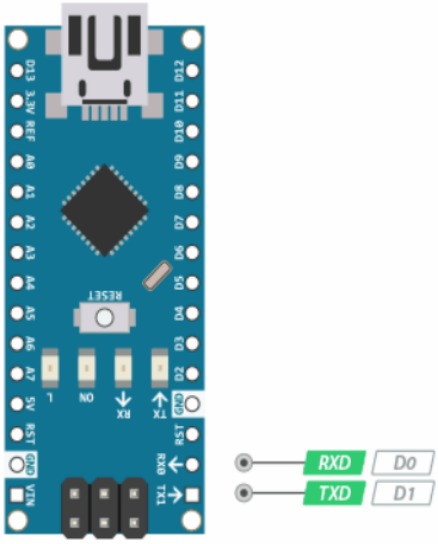
- **D11:**

    - **Function:** This serves as the outbound data pathway. The Arduino uses this pin to transmit specific commands and data *to* the targeted sensor or module.
- **D12: MISO (Master In Slave Out)**
    - **Function:** This serves as the inbound data pathway. The peripheral hardware uses this line to relay requested information or sensor values *back* to the microcontroller.
- **D13: SCK (Serial Clock)**
    - **Function:** As a *synchronous* protocol, SPI relies on a clock signal to maintain perfect timing between devices. The Arduino emits a continuous pulse so the receiver knows precisely when to sample the data bits.
- **D10: SS (Slave Select)**
    - **Function:** This pin functions as a hardware "enable" switch to address and activate specific devices on the bus.

UART (Serial Communication) Pins

1. Hardware UART

- **The Pins: D0 (RX)** and **D1 (TX)**.
- **Function:** RX is used to Receive serial data, and TX is used to Transmit serial data.
- **Shared Connection (Crucial Warning):** This single hardware UART interface is hardwired directly to the board's USB-to-Serial converter. When you upload a sketch or open the Serial Monitor, these are the pins doing the work. **Always disconnect any external modules from D0 and D1 before uploading code**, otherwise the upload will fail due to data collision.



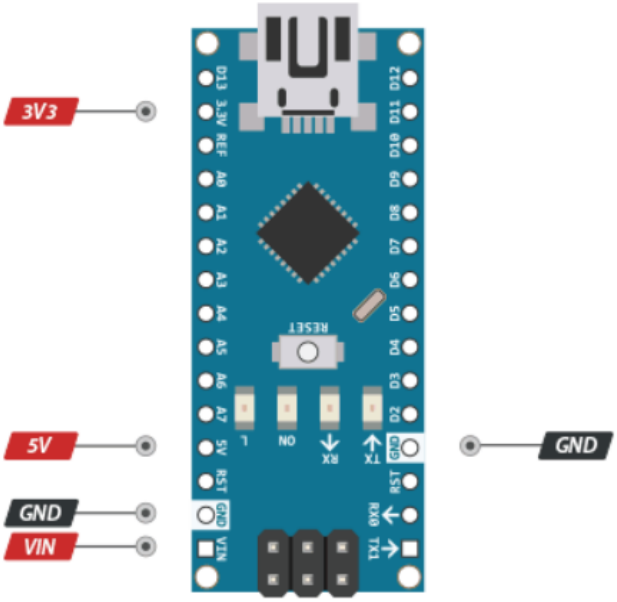
2. The SoftwareSerial Alternative

- **The Solution:** If your project requires more than one serial device (for example, reading a GPS module while simultaneously sending data to a Bluetooth module), you can use the built-in `SoftwareSerial` library.
- **How it Works:** It allows you to configure standard digital pins to emulate UART communication.
- **The Catch:** Software-based UART is notably slower and less reliable than the dedicated hardware UART, so use it only when necessary and at lower baud rates.

Power Pins

1. Powering the Board (Input)

- **VIN:** Use this pin to power the Nano externally (e.g., with a battery pack or wall adapter) when it is disconnected from USB.
    - **Voltage Range:** The recommended input is **7V to 12V**. The onboard voltage regulator safely steps this down to the 5V needed by the microcontroller.



2. Powering Components (Output)

- **5V Pin:** Delivers a regulated 5V to external sensors and components.
    - **Current Limits:** If powering the Nano via USB, you can draw a maximum of **~500mA**. If powered via VIN, the onboard regulator is technically rated for

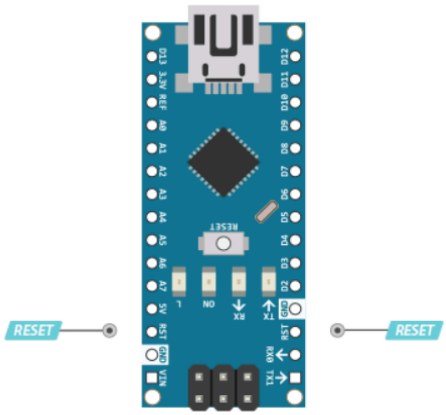
800mA. However, due to thermal limitations (the chip gets very hot), you should strictly keep the draw between **400mA to 500mA**. If your project needs more power (like driving multiple servo motors), you must use a separate power supply.

- **3.3V Pin:** Delivers a regulated 3.3V for low-voltage components (like SD card adapters or specific radio modules).
    - **Current Limits:** Exercise caution—this pin has a very low maximum capacity of approximately **50mA**.
- **GND:** The common ground reference. All external components must have their ground connected back to one of the Nano's GND pins to complete the electrical circuit.

Reset Pin (RST)

1. Overview & Logic

- **What it does:** This pin provides the capability to force a hardware restart of the ATmega328P microcontroller, effectively re-initializing your sketch from the starting line of code.
- **Active LOW:** The pin utilizes an integrated pull-up resistor to maintain a default HIGH logic state. To initiate the reset sequence, the signal must be pulled LOW (0V).



2. How to Trigger It

- **Hardware Trigger:** You can implement an external physical interface by wiring a momentary push-button between the RST and GND pins. Engaging the switch completes the circuit to ground, triggering an immediate reboot—a feature highly recommended for projects housed in custom enclosures.
- **Software Trigger:** The Arduino can be configured to reset programmatically. By bridging the RST pin to a standard digital I/O pin, you can command a reset by driving that specific pin LOW within your code using digitalWrite(pin, LOW).